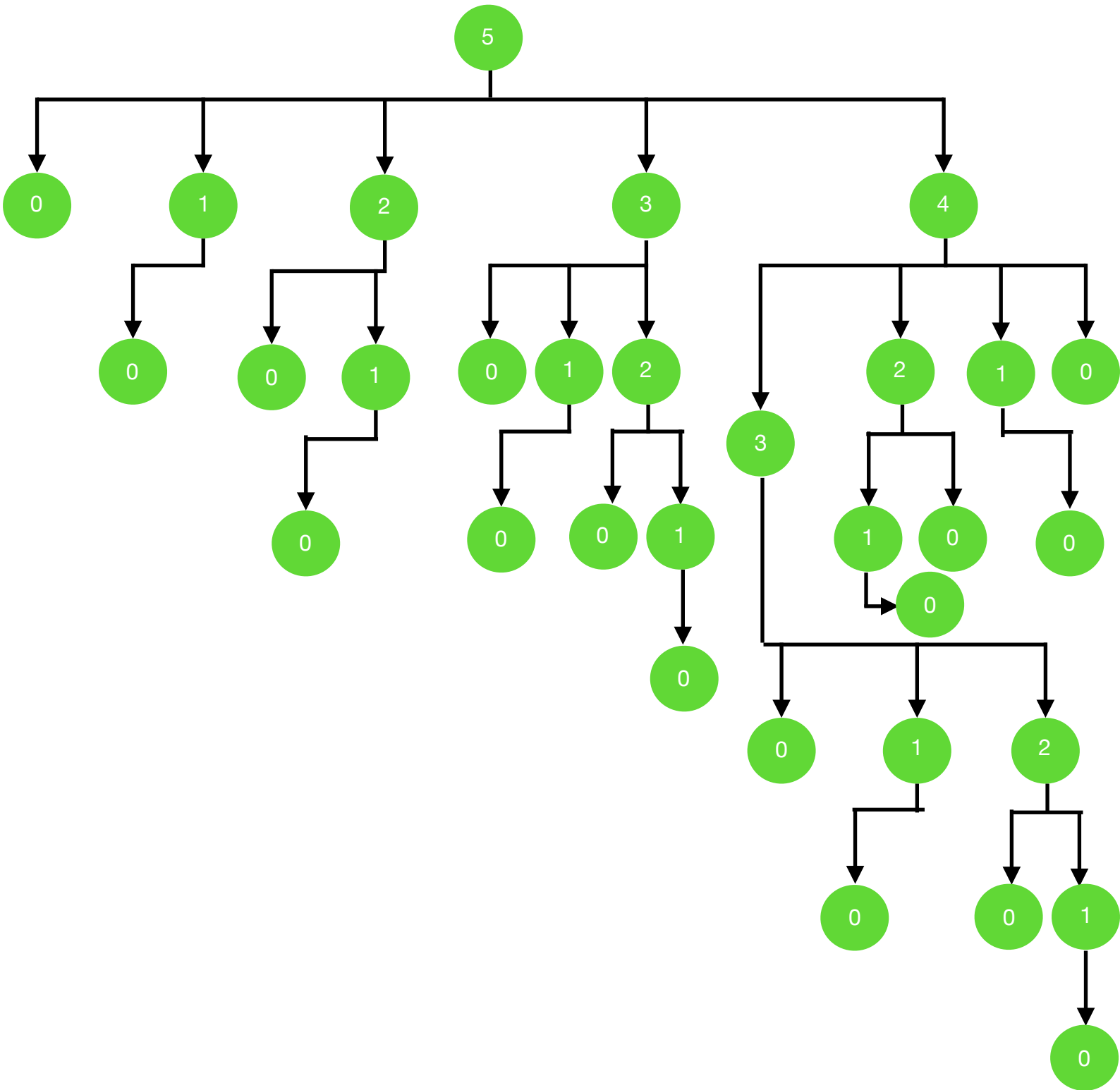


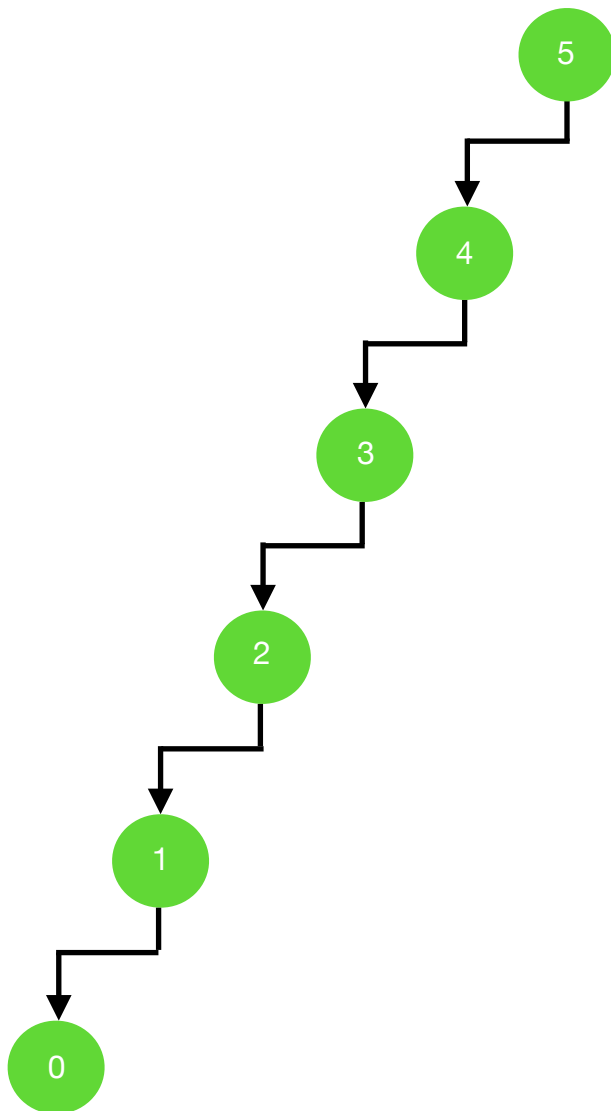
תרגיל בית 4:

שאלה 2:

ג. עץ הרקורסיה של $profit$ עבור $size = 5$ (התהפך בהעברה - מימין לשמאל):



||. עץ הרקורסיה של $profit2$ עבור $size = 5$:



טבלת זמני הריצה של שלושת הפונקציות - profit, profit2, profit3 עבור קלט size של 25 ו-50 (נכתב בפורום כי מותר להתייחס לסעיף זה כאשר $size = 25$):

	profit	profit2	profit3
Size = 25	21.161784	0.00023828	0.01052434
Size = 50	-יותר מידי-	0.00079557	1.455976

יש מספר סיבות להבדלים מהותיים אלה בין זמני הריצה של שלושת הפונקציות הקשורים באופן פעולת הפונקציות.

ראשית, הפונקציה profit פועלת על ידי ביצוע size קריאות רקורסיביות עבור כל קלט size. זה משום שבפונקציה אנו עוברים על לולאה מ-0 ועד size (לא כולל) באמצעות i ובכל איטרציה קוראים רקורסיבית לפונקציה עם קלט של $size - i - 1$. כלומר, בסה"כ מתבצעות עבור כל קלט size מתבצעות הקריאות הרקורסיביות עבור קלט של $size - 1$ ועוד אחד. למשל, עבור קלט של $size = 5$ מתבצעות בסה"כ 31 קריאות רקורסיביות באמצעות profit.

לעומת זאת, profit2 משתמשת בטכניקה הכנויה ממואיזציה על מנת לבטל את החזרתיות בקריאות רקורסיביות עם קלט שכבר חושב עבורו באמצעות שמירת הפלט של הפונקציה עבור מזהה קלט מסוים ובכך מניעת קריאה רקורסיבית חוזרת עם קלט זהה. בכך, במימוש שכזה במקרה המובא לנו בשאלה, אנו מסוגלים לעבור על כל קלט מ-size ועד 0 ושמירתו במילון מתאים ובכל קריאה רקורסיבית לבדוק את הקומבינציה שתביא את הרווח המקסימלי באמצעות לקיחת הערך המתאים מהמילון במקום לבצע קריאה רקורסיבית נוספת (כמו ב-profit). למשל, עבור קלט של $size = 5$ מתבצעות בסה"כ 5 קריאות רקורסיביות ובכך אנו לא חוסכים רק את הקריאות הרקורסיביות עצמן לפונקציה אלא גם את החישובים והפקודות המתבצעות בכל קריאה מתאימה. בדיוק בשל סיבה זו, זמן הריצה של profit2 קצר הרבה יותר משל profit.

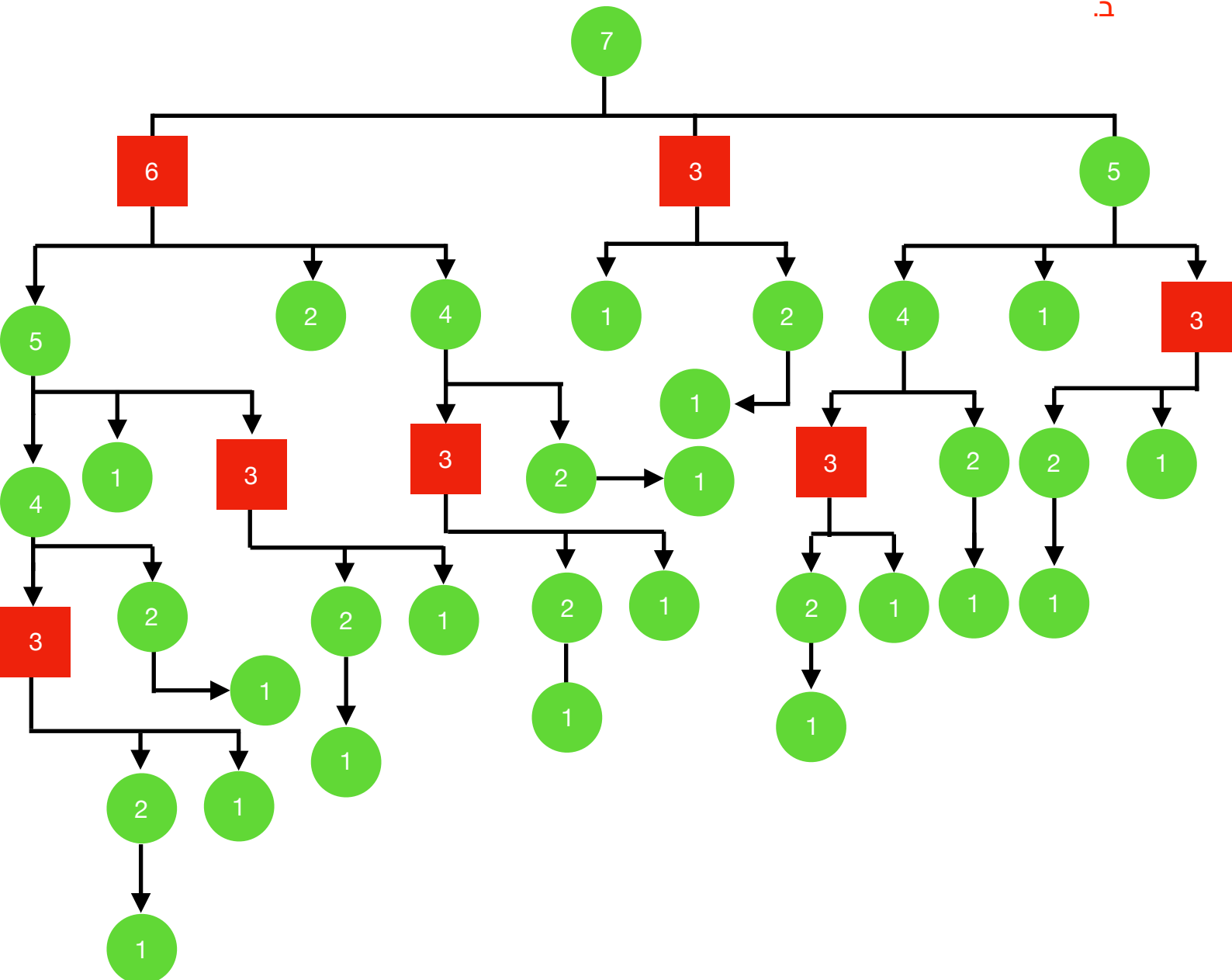
יתרה מכך, profit3 פועל בצורה שונה. לעומת profit2, הוא אינו משתמש בממואיזציה ובניגוד ל-profit שיעבור על כל התמורות של כל סידור אפשרי עבור קלט value ו-size נתונים, ולפי האפשרות יבצע קריאה רקורסיבית, profit3 יעבור רק על תמורות אפשריות מסודרות מתאימות ולכן נחסכות קריאות רקורסיביות רבות (והרי שאפילו הקריאה הרקורסיבית עצמה דורשת משאבים) והפעולות המתאימות באותן קריאות רקורסיביות אך לא נחסכות קריאות רקורסיביות עד כדי size קריאות רקורסיביות (כמו ב-profit2).

לפיכך, קצב הגידול של זמן הריצה עבור קלט גדול פי 2 (למשל במעבר מ- $size = 25$ ל- $size = 50$), יהיה הכי קרוב לליניארי עבור profit2, אחריו profit3 (אך עדיין רחוק מאוד) ואחרון profit שעבור קלט של $size = 50$ מרוב אי יעילותו לא היה מספיק זמן כדי לחשבו. ב-profit בכל הריצה קוראים ל- n^n קריאות רקורסיביות (עד כדי קבוע) ואילו ב-profit3 כל הריצה תגרוך בדיוק 2 קריאות רקורסיביות.

ולכן קיימים ההבדלים בזמני הריצה המוצגים בטבלה למעלה.

שאלה 3:

ב.



כפי שניתן לראות בעץ הרקורסיה וכמובן שבמימוש הפונקציה שלי בקובץ הפייתון, נמנעתי מכניסה לקריאות רקורסיביות כשאין צורך למשל על ידי בדיקה האם החסרת המהלך מהמספר נותנת אפס או מספר שלילי (ולכן מצב מנצח כמובן) ובכך חסכתי מזמן ריצת הפעולה והעבודה הנדרשת לעשות מכך.

הדבר מתבטא בעץ הרקורסיה בכך שהקריאות הרקורסיביות עבור 1 מתייחסות כמצב מנצח, או לצורך העניין כל "קריאה רקורסיבית" (שנחסכה על ידי הבדיקה) למספר 0 או שלילי ובכך מתקבל מצב מנצח לקורא לו.

שאלה 4:

א.

כאשר ישנם 3 משתתפים (כלומר, $N = 3$), כל משתתף בהפעלת הפרוטוקול המתואר מבצע 4 פעולות של modular exponentiation. למשל, עבור משתתפת Alice, הפעולות הבאות מבוצעות:

- I. מחשבת את g^a .
- II. מחשבת את g^{b^a} לאחר קבלת g^b .
- III. מחשבת את g^{c^a} לאחר קבלת g^c .
- IV. מחשבת את $(g^{b^c})^a$ לאחר קבלת g^{b^c} .

כעת יש ל-Alice את המפתח המשותף (g^{a*b*c}) והפרוטוקול הושלם. והרי כי שאר המשתתפים מבצעים את הפעולות המתאימות עבורם (אותו מספר פעולות).

ב.

כעת אתאר פרוטוקול שעל פיו כל משתתף מבצע רק 3 פעולות של modular exponentiation. אתאר את הפרוטוקול באמצעות שימוש ב-3 משתתפים כלליים - Alice, Bob, Carol. על מנת לעקוב אחר אופן פעולת הפרוטוקול אשתמש באות הראשונה של כל שם כדי לתאר מי מבצע כל פעולה ואמספר אותה בהתאם:

- A - מחשבת את g^a ושולחת אותו ל-B ו-C (פעולה 1).
- B - מחשב את g^b ושולח אותו ל-A ו-C (פעולה 1).
- C - מחשבת את g^{a^c} ואת g^{b^c} ושולחת את g^{a^c} ל-B ואת g^{b^c} ל-A. (פעולה 1,2).
- A - מחשבת את g^{b^a} ושולחת ל-C (פעולה 2).
- B - מחשב את g^{a^b} (פעולה 2).
- A - מחשבת את $(g^{b^c})^a$ (פעולה 3).
- B - מחשב את $(g^{a^c})^b$ (פעולה 3).
- C - מחשבת את $(g^{a^b})^c$ (פעולה 3).

כעת ל-Alice, Bob, Carol יש את המפתח המשותף g^{a*b*c} והרי שכל אחד מהם ביצע 3 פעולות של modular exponentiation כדרוש.

ג.

1.

משתתף 1 ישלח למשתתף 2 בשלב האחרון את ההודעה שתכיל את $g^{a_5 * a_6 * a_7 * a_8 * a_4 * a_3 * a_1}$.

2.

משתתף 2 בשלב האחרון יחשב את $(g^{a_5 * a_6 * a_7 * a_8 * a_4 * a_3 * a_1})^{a_2}$ על מנת להגיע למפתח המשותף לכל המשתתפים.

3.

המפתח המשותף בין שמונת המשתתפים יהיה $g^{a_1 * a_2 * a_3 * a_4 * a_5 * a_6 * a_7 * a_8}$.

4.

עבור N משתתפים בפרוטוקול, יבוצעו פעולות modular exponentiation בסדר גודל של $O(\log_2 N)$. כפי שראינו עבור 8 משתתפים, מספר הפעולות שכל משתמש מבצע יהיה שווה למספר הפעמים בהם היינו צריכים לחלק את קבוצת המשתתפים ב-2 על מנת להגיע בסופו של דבר למשתתף יחיד. זה משום, שבכל שלב המשתתף מבצע פעולה אחת שכזו (מחשב את מה שקיבל בחזקת מספרו האישי ושולח) והוא עושה פעולה זו מההתחלה עד השלב האחרון בו כדי לקבל את המפתח המשותף זה לעלות את המספר שנשלח לו בחזקת מספרו האישי. לפיכך, בכל שלב מבצע פעולה אחת שכזו והרי כי מספר השלבים הוא מספר החלוקות ב-2 והרי שהוא $\log_2 N$.

לכן מספר פעולות ה-modular exponentiation שייעשו הינם מסדר גודל של $O(\log_2 N)$.

שאלה 5:

נניח כי מספר הביטים בייצוג הבינארי של a הינו n ומספר הביטים בייצוג הבינארי של b הינו m . כנדרש בשאלה, ננתח את סיבוכיות זמן הריצה במקרה הגרוע ביותר. במקרה זה, כל הביטים של מספר a (כלומר, בייצוג בינארי זה n אחדים) וזה משום שאז בפעולות החיבור והכפל ה- $carry$ הוא המקסימלי ומספר הפעולות שנבצע בביצוע אריתמטיקה יהיה הגדול ביותר ולכן גם תוצר הכפל של המספר יהיה בעל מספר הביטים המקסימלי (נתייחס לכך בהמשך). התייחסנו רק לביטים המופעלים של a מפני שכפי שנסביר בהמשך, פעולת הכפל של a היא המשמעותית ובמציאת החסם נתייחס אליה. בסה"כ בפעולת הכפל יש $n * n$ פעולות כפל ופעולות החיבור הנלוות לחישוב הכפל זניחות משום שהוא לא יכול להיות יותר מ- $n + 1$ פעולות חיבור.

בפעולה $power$ המממשת חזקה באמצעות שיטה של iterated squaring אנו עושים שימוש בלולאה המסתיימת כאשר עברנו על כל הביטים של הביטים של b (מפני שהיא רצה כל עוד b חיובי ואנחנו משנים את b רק באמצעות חלוקה בשתיים ועיגול מטה - פעולה השקולה להעתקת b ללא הביט הימני ביותר שלו), כלומר, אנו מבצעים m איטרציות.

בכל איטרציה אנו בודקים האם b אי זוגי או זוגי, כלומר, בדיקה האם הביט הימני ביותר של b מופעל או לא ובדיקה זו רצה בזמן $O(1)$. כמו כן, בכל איטרציה אנו מבצעים את הפעולות הבאות:

- I. פעולת הכפל $a * a$.
- II. פעולת החלוקה $b // 2$ - פועלת בזמן ריצה של $O(b)$.
- III. פעולה הכפל $result * a$ (אם b אי זוגי).

ניתן להזניח את פעולת הכפל $result * a$ משום שתמיד $a * a > result * a$ בכל איטרציה שכן אנו מכפילים את a בעצמו בכל איטרציה ומכפילים את $result$ (שבהתחלה = 1) ב- a רק כאשר b אי זוגי (ומפני שהוא מתחיל ב-1) אזי $a * a > result * a$ וידוע כי a, b שלמים חיוביים ובשל שאנו מחפשים חסם ניתן להזניח את הפעולה $result * a$. כמו כן, הפעולות המתבצעות על b (בדיקת הלולאה והחלוקה) הינן ליניאריות או תלויות קבוע ולכן ניתן להזניח אותן בחיפוש אחר החסם ההדוק ביותר.

כעת נתייחס לפעולת הכפל המשמעותית לפיה נחשב סיבוכיות הפונקציה: $a * a$. במקרה הגרוע ביותר, אותו כבר תיארתי, כל הביטים של a (n) מופעלים ולפיכך תוצאת הכפל תהיה הכפלת מספר הביטים של n (למספר בעל 2^n) וחוזר חלילה m פעמים. הרי כי סיבוכיות זמן הריצה של הכפולה הזאת מסדר גודל של $O(n^2)$ וזה משום שניתן להתייחס לכפל של המספר ככפל של כל ביט ב- a בכפל בכל ביט ב- a ואלה הרי n ולכן כל פעולת כפל היא מסדר גודל של כפולת מספר הביטים. נחבר את שתי המסקנות האחרונות אליהן הגענו על מנת לחשב את סה"כ סיבוכיות זמן הריצה של הפעולה באמצעות חיבור מה שקורה בכל איטרציה, כלומר:

$$n * n + (2n) * (2n) + (4n) * (4n) + \dots + (2^{m-1}n) * (2^{m-1}n) = n^2 + 4n^2 + 16n^2 + \dots + 2^{2(m-1)}n^2 = n^2 * (1 + 4 + 16 + \dots + 4^{m-1}) =$$

על פי נוסחאת סכום סדרה הנדסית, כאשר האיבר הראשון הוא 1 ומתקדמים בכפולות של 4:

$$n^2 * \frac{4^m - 1}{3} = \frac{1}{3} * (n^2 * 4^m - 1) \leq c * (4^m * n^2) = O(4^m * n^2)$$

ולפיכך, סיבוכיות זמן הריצה של הפעולה a^b בשיטה זו כתלות ב- m (מספר הביטים ב- b) וב- n (מספר הביטים ב- a) הינה $O(4^m * n^2)$.

שאלה 6:

ג.

בשאלה רשום לצרף את ההרצה, אך הוספתי גם טבלה של ההדפסה.

```
>>> for i in range(7,17):
      print(rw_stats(2**i, 1, number_runs=10**3))

(128, 8.918, 0.7882472843277037, 0.0, 38.0, 0.93)
(256, 12.572, 0.78575, 0.0, 50.0, 0.95)
(512, 17.906, 0.7913408764953949, 0.0, 72.0, 0.966)
(1024, 24.976, 0.7805, 0.0, 102.0, 0.977)
(2048, 37.296, 0.824132953472921, 0.0, 146.0, 0.984)
(4096, 52.012, 0.8126875, 0.0, 218.0, 0.985)
(8192, 69.74, 0.7705254206242159, 0.0, 288.0, 0.99)
(16384, 99.368, 0.7763125, 0.0, 470.0, 0.996)
(32768, 142.058, 0.7847669931390512, 0.0, 534.0, 0.995)
(65536, 203.874, 0.7963828125, 0.0, 874.0, 0.997)
```

n	<i>average dist</i>	$\frac{\text{average dist}}{\sqrt{\text{runlength}}}$	<i>min dist</i>	<i>max dist</i>	<i>origin freq</i>
2^7	8.918	0.788	0	38	0.93
2^8	12.572	0.785	0	50	0.95
2^9	17.906	0.791	0	72	0.966
2^{10}	24.976	0.780	0	102	0.977
2^{11}	37.296	0.824	0	146	0.984
2^{12}	52.012	0.812	0	218	0.985
2^{13}	69.74	0.770	0	288	0.99
2^{14}	99.368	0.776	0	470	0.996
2^{15}	142.058	0.782	0	534	0.995
2^{16}	203.874	0.796	0	874	0.997

1.

המרחק הממוצע מהראשית כפונקציה של מספר הצעדים n בהילוך הוא $\sqrt{\frac{2 \cdot n}{\pi}}$ (אין צורך להוכיח).

2.

ההסתברות שהילוך באורך n המתחיל בראשית יעבור שוב בראשית היא 1. משום שכפי שניתן לראות בהדפסת התוצאות המצורפות, ככל שמספר ההרצות גדול יותר כך גם שכיחות החזרה לראשית שואפת ל-1 באופן מובהק. כמו כן, על מנת לחזור לראשית, על כל הקואורדינטות להתאפס ובכל צעד אנו מבצעים התקדמות של ± 1 באופן רנדומלי. יש רק קואורדינטה יחידה עבור מימד 1 ולכן מרכיב יחיד שמשפיע על הקרבה לראשית ולכן על הצעד להתקדם רק בכיוון אחד (חיובי או שלילי) וכאשר יש סיכוי של חצי לקבל צעד "קדימה" או "אחורה" נשער שבביצוע צעדים רבים נתרכז לרוב סביב נקודת האפס וכמובן שנחזור אליה.

Dimension = 2

n	<i>average dist</i>	$\frac{\text{average dist}}{\sqrt{\text{runlength}}}$	<i>min dist</i>	<i>max dist</i>	<i>origin freq</i>
2^7	14.127	1.248	0	44.944	0.595
2^8	20.078	1.254	0	63.529	0.616
2^9	28.054	1.239	2	88.204	0.656
2^{10}	39.591	1.237	2	118.152	0.704
2^{11}	56.697	1.252	2	176.408	0.697
2^{12}	78.172	1.221	4	239.232	0.731
2^{13}	112.920	1.247	2.828	327.200	0.724
2^{14}	164.204	1.282	4	465.553	0.757
2^{15}	222.188	1.227	11.661	714.943	0.772
2^{16}	321.049	1.254	11.313	949.766	0.753

Dimension = 3

n	<i>average dist</i>	$\frac{\text{average dist}}{\sqrt{\text{runlength}}}$	<i>min dist</i>	<i>max dist</i>	<i>origin freq</i>
2^7	18.032	1.593	0	51.380	0.281
2^8	25.822	1.613	2.828	75.392	0.267
2^9	37.113	1.640	4.472	85.252	0.263
2^{10}	50.349	1.573	2.828	136.557	0.282
2^{11}	71.936	1.589	4.898	187.467	0.249
2^{12}	102.163	1.596	4	268.700	0.288
2^{13}	144.120	1.592	8.944	422.700	0.27
2^{14}	199.155	1.555	16.613	499.031	0.282
2^{15}	285.883	1.579	22	709.100	0.286
2^{16}	411.868	1.608	35.832	991.832	0.264

Dimension = 4

n	$average\ dist$	$\frac{average\ dist}{\sqrt{runlength}}$	$min\ dist$	$max\ dist$	$origin\ freq$
2^7	21.466	1.897	2.828	45.431	0.111
2^8	29.729	1.858	4.898	74.296	0.123
2^9	43.400	1.918	4.898	113.066	0.107
2^{10}	60.275	1.883	8.485	146.615	0.103
2^{11}	85.549	1.890	15.231	205.260	0.112
2^{12}	120.745	1.886	18.0	305.018	0.107
2^{13}	171.524	1.895	38.418	407.641	0.093
2^{14}	239.153	1.868	38.522	548.375	0.108
2^{15}	344.268	1.901	63.655	832.091	0.111
2^{16}	469.996	1.835	58.923	1092.205	0.121

1.

שכיחות החזרה לראשית כפונקציה של המימד יורדת כפי שניתן לראות בטבלאות המצורפות - עבור מימד 1 הרי ששכיחות החזרה מתקרבת ל-1, עבור מימד 2 שכיחות החזרה מגיעה לסביבות 0.75, עבור מימד 3 שכיחות החזרה נעה בין 0.28 ל-0.26 ועבור מימד 4 שכיחות החזרה היא בממוצע 0.110. קיבלנו תוצאות אלה משום שעל מנת לחזור לראשית, על כל הקואורדינטות להתאפס וככל שיש יותר קואורדינטות ובכל צעד אנו מבצעים התקדמות של $+1/-1$ באופן רנדומלי אזי הסיכוי של כל הקואורדינטות להתאפס קטן. למשל, עבור מימד 1, יש רק קואורדינטה אחת ולכן מרכיב יחיד שמשפיע על הקרבה לראשית ולכן על הצעד להתקדם רק ב"ציר" אחד (חיובי או שלילי) וכאשר יש סיכוי של חצי לקבל צעד "קדימה" או "אחורה" נשער שבביצוע צעדים רבים נתרכז לרוב סביב נקודת האפס. לעומת זאת, ככל שיש יותר מימדים, יש יותר קואורדינטות ולכן מרכיבים רבים יותר אשר על כולם להתרכז באפס (בו זמנית!) על מנת שהוא ישוב לראשית.

עצם ההישענות על כך שיש מספר מרכיבים שבאופן רנדומלי צריכים כולם להיות שווים לאפס בו זמנית מסבירה לנו את התנהגדות הפונקציה של שכיחות החזרה לראשית כפונקציה של המימד - יורדת מהר.

2.

התוצאות שקיבלנו עבור מימדים גדולים מ-1 תואמות את כך שממוצע המרחק פרופורציוני לשורש מספר הצעדים (בקירוב - יש נוסחא מדויקת שקראתי עליה אך לא צירפתי אותה עבור מימדים גדולים מ-1). ניתן לכך מספר דוגמאות - עבור מימד 2: $\sqrt{n^{16}} = 2 * \sqrt{n^{14}}$ וגם $321.5 \sim 2 * 164$ (מרחקים ממוצעים מתאימים), עבור מימד 3: $\sqrt{n^{16}} = 2 * \sqrt{n^{14}}$ וגם $410 \sim 2 * 199.12$, עבור מימד 4: $\sqrt{n^{16}} = 2 * \sqrt{n^{14}}$ וגם $469 \sim 2 * 239$.

לפיכך, הראיתי כי אכן המרחק הממוצע פרופורציוני לשורש מספר הצעדים.